MODULE 3

Memory management - Address Space, Memory API, Address Translation - An Example, Dynamic (Hardware-based) Relocation, Segmentation: Generalized Base/Bounds, Address translation in segmentation, Support for Sharing.

Virtual memory - Paging: Introduction, page tables and hardware support, TLBs, Example: Accessing an Array, - TLB hits and misses, Handling TLB misses, TLB structure, Reducing the page table size.

Going beyond physical memory - Swap space, page fault and its control flow, page replacement policies, Thrashing.

****

MEMORY MANAGEMENT

The memory is central to the operation of a modern computer system.

- Memory consists of a large array of bytes, each with its own address.
- The CPU fetches instructions from memory according to the value of the program counter.
- These instructions may cause additional loading from and storing to specific memory addresses.
- A typical instruction-execution cycle, for example, first fetches an instruction from memory.
- The instruction is then decoded and may cause operands to be fetched from memory.
- After the instruction has been executed on the operands, results may be stored back in memory.
- The memory unit sees only a stream of memory addresses; it does not know how they are generated (by the instruction counter, indexing, indirection, literal addresses, and so on) or what they are for (instructions or data).

Basic Hardware

- Main memory and the registers built into the processor itself are the only general-purpose storage that the CPU can access directly.
- There are machine instructions that take memory addresses as arguments, but none that take disk addresses.
- Therefore, any instructions in execution, and any data being used by the instructions, must be in one of these direct-access storage devices. If the data are not in memory, they must be moved there before the CPU can operate on them.
- Registers that are built into the CPU are generally accessible within one cycle of the CPU clock.
- But completing a memory access may take many cycles of the CPU clock. In such cases, the processor normally needs to stall, since it does not have the data required to complete the instruction that it is executing.
- This situation is intolerable because of the frequency of memory accesses. The remedy is to add fast memory between the CPU and main memory, typically on the CPU chip for fast access.

CONCEPT OF ADDRESS SPACES

Earlier, when the use of machines increased and because machines were expensive, people began to share machines more effectively. Thus, the era of multiprogramming was born, in which multiple processes were ready to run at a given time, and the OS would switch between them, for example when one decided to perform an I/O. Doing so, increased the effective utilization of the CPU. Such increases in efficiency were particularly important in those days and the concept of time sharing was born.

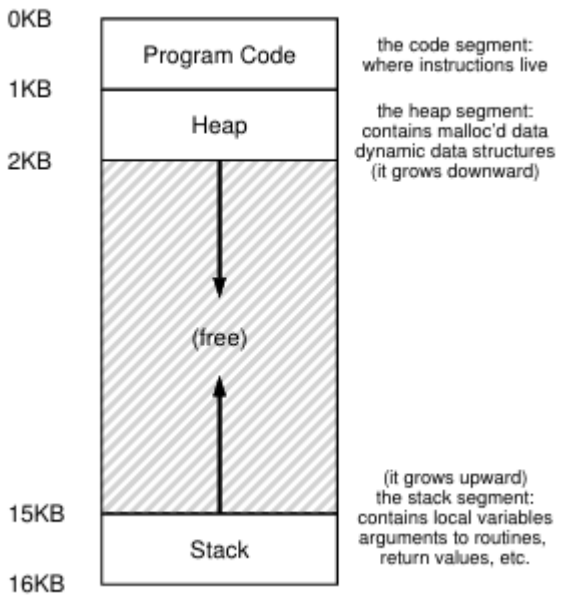


Figure 3.1: An Example - Address Space

The address space of a process contains all of the memory state of the running program. For example, the **code** of the program (the instructions) have to live in memory somewhere, and thus they are in the address space. The program, while it is running, uses a **stack** to keep track of where it is in the function call chain as well as to allocate local variables and pass parameters and return values to and from routines. Finally, the **heap** is used for dynamically-allocated, user-managed memory, like a call to malloc() in C or new in an object oriented language such as C++ or Java.

In the example in Figure 3.1, we have a tiny address space (only 16 KB). The program code lives at the top of the address space (starting at 0 in this example, and is packed into the first 1K of the address space). Code is static (and thus easy to place in memory), so we can place it at the top of the address space and know that it won't need any more space as the program runs. Next, we have the two regions of the address space that may grow (and shrink) while the program runs. Those are the heap (at the top) and the stack (at the bottom). We place them like this because each wish to be able to grow, and by putting them at opposite ends of the address space, we can allow such growth: they just have to grow in opposite directions. The heap thus starts just after the code (at 1KB) and grows downward (say when a user requests more memory via malloc ()); the stack starts at 16KB and grows upward (say when a user makes a procedure call).

When the OS does this, we say the **OS is virtualizing memory**, because the running program thinks it is loaded into memory at a particular address (say 0) and has a potentially very large address space (say 32-bits or 64-bits); the reality is quite different.

Goals of Virtual Memory (VM)

- One major goal of a virtual memory (VM) system is transparency. The OS should implement virtual memory in a way that is invisible to the running program. Thus, the program shouldn't be aware of the fact that memory is virtualized; rather, the program behaves as if it has its own private physical memory. Behind the

scenes, the OS (and hardware) does all the work to multiplex memory among many different jobs, and hence implements the illusion.

- Another goal of VM is efficiency. The OS should strive to make the virtualization as efficient as possible, both in terms of time and space. In implementing time-efficient virtualization, the OS will have to rely on hardware support, including hardware features such as TLBs.
- Finally, a third VM goal is protection. The OS should make sure to protect processes from one another as well as the OS itself from processes. When one process performs a load, a store, or an instruction fetch, it should not be able to access or affect in any way the memory contents of any other process or the OS itself (that is, anything outside its address space). Protection thus enables us to deliver the property of isolation among processes; each process should be running in its own isolated cocoon, safe from the ravages of other faulty or even malicious processes

MEMORY API

Types of Memory

In running a C program, there are two types of memory that are allocated. The first is called **stack** memory, and allocations and deallocations of it are managed implicitly by the compiler for the programmer; for this reason, it is sometimes called automatic memory.

```
void func()
{
int x;
// declares an integer on the stack ...
 }
```

It is this need for long-lived memory that gets us to the second type of memory, called **heap** memory, where all allocations and deallocations are explicitly handled by the programmer.

```
void func()
{
 int *x = (int *) malloc(sizeof(int));
 ...
 }
```

The malloc() Call

When asking for some room on the heap, the malloc() call either succeeds and gives you back a pointer to the newly-allocated space, or fails and returns NULL.

```
void *malloc(size_t size);
```

The free() Call

To free heap memory that is no longer in use, programmers simply call free():

```
 int *x = malloc(10 * sizeof(int));
...
free(x);
```

The routine takes one argument, a pointer that was returned by malloc().The size of the allocated region is not passed in by the user, and must be tracked by the memory-allocation library itself.

Common Errors

There are a number of common errors that arise in the use of malloc() and free().

- Forgetting To Allocate Memory: Leads to segmentation fault.
- Not Allocating Enough Memory: A related error is not allocating enough memory, sometimes called a buffer overflow.
- Forgetting to Initialize Allocated Memory: If you do forget, your program will eventually encounter an uninitialized read, where it reads from the heap some data of unknown value.
- Forgetting To Free Memory: Another common error is known as a memory leak, and it occurs when you forget to free memory. In long-running applications or systems (such as the OS itself), this is a huge problem, as slowly leaking memory eventually leads one to run out of memory, at which point a restart is required.
- Freeing Memory Before You Are Done With It: Sometimes a program will free memory before it is finished using it; such a mistake is called a dangling pointer.
- Freeing Memory Repeatedly: Programs also sometimes free memory more than once; this is known as the double free.
- Calling free() Incorrectly: free() expects only to pass to it one of the pointers that received from malloc() earlier. The invalid frees are dangerous and of course should be avoided.

ADDRESS TRANSLATION

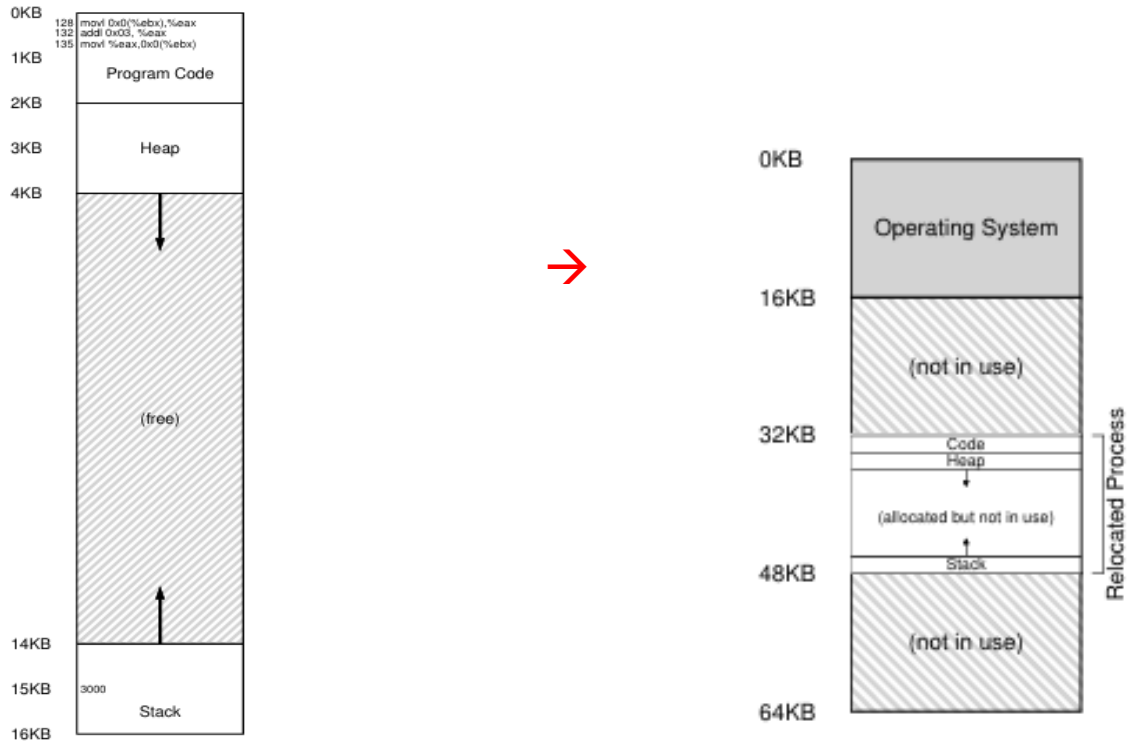
In developing the virtualization of the CPU, a general mechanism known as **Limited Direct Execution** (or LDE) is done: for the most part, let the program run directly on the hardware; however, at certain key points in time (such as when a process issues a system call, or a timer interrupt occurs), arrange so that the OS gets involved and makes sure the "right" thing happens. Thus, the OS, with a little hardware support, tries its best to get out of the way of the running program, to deliver an efficient virtualization; however, by interposing at those critical points in time, the OS ensures that it maintains control over the hardware. Efficiency and control together are two of the main goals of any modern operating system.

The generic technique that can be added to the general approach of limited direct execution, is something that is referred to as **hardware-based address translation**, or just address translation for short. **With address translation, the hardware transforms each memory access (e.g., an instruction fetch, load, or store), changing the virtual address provided by the instruction to a physical address where the desired information is actually located**. Thus, on each and every memory reference, an address translation is performed by the hardware to redirect application memory references to their actual locations in memory. Of course, the hardware alone cannot virtualize memory, as it just provides the low-level mechanism for doing so efficiently. The OS must get involved at key points to set up the hardware so that the correct translations take place; it must thus manage memory, keeping track of which locations are free and which are in use, and judiciously intervening to maintain control over how memory is used. The goal of all of this work is to create a beautiful illusion: that the program has its own private memory, where its own code and data reside. Behind that virtual reality lies the ugly physical truth: that many programs are actually sharing memory at the same time, as the

CPU (or CPUs) switches between running one program and the next. Through virtualization, the OS (with the hardware's help) turns the ugly machine reality into something that is a useful, powerful, and easy to use abstraction.

AN EXAMPLE:

However, to virtualize memory, the OS wants to place the process somewhere else in physical memory, not necessarily at address 0. Thus, we have the problem: how can we relocate this process in memory in a way that is transparent to the process? How can provide the illusion of a virtual address space starting at 0, when in reality the address space is located at some other physical address?



A Process And Its Address Space

Physical Memory with a Single Relocated Process

Figure 3.2 Example: Virtualizing Memory

The OS decides to load the process that is at address 0 to the physical address 32 KB.

DYNAMIC (HARDWARE-BASED) RELOCATION

Specifically, we'll need two hardware registers within each CPU: one is called the **base** register, and the other the **bounds** (sometimes called a **limit** register). This base-and-bounds pair is going to allow us to place the address space anywhere we'd like in physical memory, and do so while ensuring that the process can only access its own address space. In this setup, each program is written and compiled as if it is loaded at address zero. However, when a program starts running, the OS decides where in physical memory it should be loaded and sets the base register to that value. In the example above, the OS decides to load the process at physical address 32 KB and thus sets the base register to this value.

Each memory reference generated by the process is a **virtual address**; the hardware in turn adds the contents of the base register to this address and the result is a physical address that can be issued to the memory system.

Transforming a virtual address into a physical address is exactly the technique we refer to as address translation; that is, the hardware takes a virtual address the process thinks it is referencing and transforms it into a physical address which is where the data actually resides. Because this relocation of the address happens at runtime, and because we can move address spaces even after the process has started running, the technique is often referred to as **dynamic relocation.**

Namely, a base register is used to transform virtual addresses (generated by the program) into physical addresses. A bounds (or limit) register ensures that such addresses are within the confines of the address space.

It should be noted that that the base and bounds registers are hardware structures kept on the chip (one pair per CPU**). The part of the processor that helps with address translation is the memory management unit (MMU).**

To manage a cache built into the CPU, the hardware automatically speeds up memory access without any operating-system control.

• Not only are we concerned with the relative speed of accessing physical memory, but we also must ensure correct operation.

• For proper system operation we must protect the operating system from access by user processes.

• On multiuser systems, we must additionally protect user processes from one another. This protection must be provided by the hardware because the operating system doesn't usually intervene between the CPU and its memory accesses. Hardware implements this protection in several different ways.

Base and Limit Registers.

We first need to make sure that each process has a separate memory space. Separate per-process memory space protects the processes from each other and is fundamental to having multiple processes loaded in memory for concurrent execution. To separate memory spaces, we need the ability to determine the range of legal addresses that the process may access and to ensure that the process can access only these legal addresses.

• We can provide this protection by using two registers, usually a base and a limit, as illustrated in Figure 3.3. The base register holds the smallest legal physical memory address; the limit register specifies the size of the range. For example, if the base register holds 300040 and the limit register is 120900, then the program can legally access all addresses from 300040 through 420939 (inclusive).

• Protection of memory space is accomplished by having the CPU hardware compare every address generated in user mode with the registers. Any attempt by a program executing in user mode to access operating-system memory or other users' memory results in a trap to the operating system, which treats the attempt as a fatal error (Figure 3.4).

• This scheme prevents a user program from (accidentally or deliberately) modifying the code or data structures of either the operating system or other users. The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction.

• A pair of base and limit registers define the logical address space.

• CPU must check every memory access generated in user mode to be sure it is between base and limit for that user.

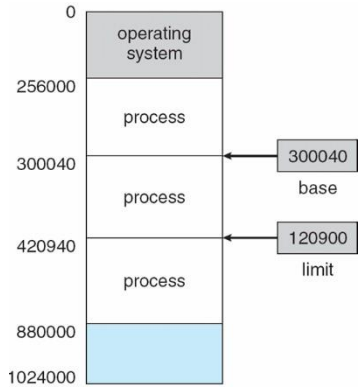


Figure 3.3 A base and a limit register that define a logical address space

Hardware Address Protection

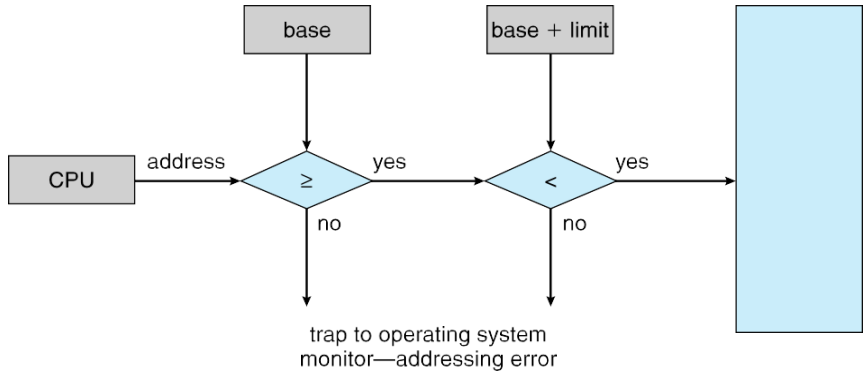


Figure 3.4 Hardware address protection with base and limit registers

OS Issues

There are a number of new OS issues that arise when using base and limit to implement a simple virtual memory. Specifically, there are three critical junctures where the OS must take action to implement this base-and-bounds(limit) approach to virtualizing memory.

- First, The OS must take action when a process is created, finding space for its address space in memory. Fortunately, given the assumptions that each address space is smaller than the size of physical memory and the same size, this is quite easy for the OS; it can simply view physical memory as an array of slots, and track whether each one is free or in use. When a new process is created, the OS will have to search a data structure (often called a free list) to find room for the new address space and then mark it used.
- Second, the OS must take action when a process is terminated, reclaiming all of its memory for use in other processes or the OS. Upon termination of a process, the OS thus puts its memory back on the free list, and cleans up any associated data structures as need be.

- Third, the OS must also take action when a context switch occurs. There is only one base and bounds register on each CPU, after all, and their values differ for each running program, as each program is loaded at a different physical address in memory. Thus, the OS must save and restore the base-and-bounds pair when it switches between processes. Specifically, when the OS decides to stop running a process, it must save the values of the base and bounds registers to memory, in some per-process structure such as the process structure or process control block (PCB). Similarly, when the OS resumes a running process (or runs it the first time), it must set the values of the base and bounds on the CPU to the correct values for this process.

MEMORY – BASIC CONCEPTS

Address Binding

Usually, a program resides on a disk as a binary executable file. To be executed, the program must be brought into memory and placed within a process.

Depending on the memory management in use, the process may be moved between disk and memory during its execution. The processes on the disk that are waiting to be brought into memory for execution form the input queue.

• Most systems allow a user process to reside in any part of the physical memory. Thus, although the address space of the computer may start at 00000, the first address of the user process need not be 00000.

• Addresses in the source program are generally symbolic. A compiler typically binds these symbolic addresses to relocatable addresses.

• **The linkage editor or loader in turn binds the relocatable addresses to absolute addresses.**

• Each binding is a mapping from one address space to another.

Classically, the binding of instructions and data to memory addresses can be done at any step along the way:

1. **Compile time**. If you know at compile time where the process will reside in memory, then absolute code can be generated. The MS-DOS programs are bound at compile time.
2. **Load time**. If it is not known at compile time where the process will reside in memory, then the compiler must generate relocatable code. In this case, final binding is delayed until load time.
3. **Execution time**. If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time. Special hardware must be available for this scheme to work. Most general-purpose operating systems use this method.

Logical vs. Physical Address Space

✓ The concept of a logical address space that is bound to a separate physical address space is central to proper memory management

- o **Logical address** – generated by the CPU; also referred to as virtual address
- o **Physical address** – address seen by the memory unit

✓ Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme

✓ Logical address space is the set of all logical addresses generated by a program

✓ Physical address space is the set of all physical addresses generated by a program

Dynamic relocation using a relocation register

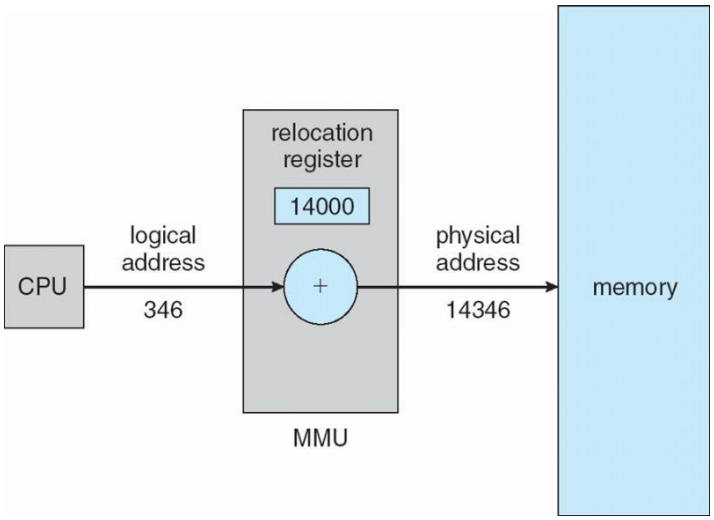


Figure 3.5 Dynamic relocation using a relocation register.

- The run-time mapping from virtual to physical addresses is done by a hardware device called the memory-management unit (MMU).
- The base register is now called a relocation register. The value in the relocation register is added to every address generated by a user process at the time the address is sent to memory (see Figure 3.5).

Dynamic Loading

To obtain better memory-space utilization, we can use dynamic loading. With dynamic loading, a routine is not loaded until it is called. All routines are kept on disk in a relocatable load format.

- The main program is loaded into memory and is executed. When a routine need to call another routine, the calling routine first checks to see whether the other routine has been loaded. If it has not, the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change.
- Then control is passed to the newly loaded routine. The advantage of dynamic loading is that a routine is loaded only when it is needed. This method is particularly useful when large amounts of code are needed to handle infrequently occurring cases, such as error routines. In this case, although the total program size may be large, the portion that is used (and hence loaded) may be much smaller.
- Dynamic loading does not require special support from the operating system. It is the responsibility of the users to design their programs to take advantage of such a method. Operating systems may help the programmer, however, by providing library routines to implement dynamic loading.

Dynamic Linking and Shared Libraries

Dynamically linked libraries are system libraries that are linked to user programs when the programs are run. Some operating systems support only static linking, in which system libraries are treated like any other object module and are combined by the loader into the binary program image.

Dynamic linking, in contrast, is similar to dynamic loading. Here, though, linking, rather than loading, is postponed until execution time. This feature is usually used with system libraries, such as language subroutine

libraries. Without this facility, each program on a system must include a copy of its language library (or at least the routines referenced by the program) in the executable image. This requirement wastes both disk space and main memory.

With dynamic linking, a stub is included in the image for each library routine reference. The stub is a small piece of code that indicates how to locate the appropriate memory-resident library routine or how to load the library if the routine is not already present. When the stub is executed, it checks to see whether the needed routine is already in memory. If it is not, the program loads the routine into memory. Either way, the stub replaces itself with the address of the routine and executes the routine. Thus, the next time that particular code segment is reached, the library routine is executed directly, incurring no cost for dynamic linking. Under this scheme, all processes that use a language library execute only one copy of the library code. This feature can be extended to library updates (such as bug fixes).

Contiguous Allocation

Contiguous memory allocation is an OS technique where each process is assigned a single, unbroken block of main memory. It requires that all parts of a process reside together, commonly using relocation and limit registers for memory protection. While simple, it suffers from internal and external fragmentation and limits the degree of multiprogramming.

- Each process gets a single, continuous segment of memory.
  - ❑ Relocation registers used to protect user processes from each other, and from changing operating- system code and data
  - ❑ Base register contains value of smallest physical address
  - ❑ Limit register contains range of logical addresses – each logical address must be less than the limit register.

Hardware Support for Relocation and Limit Registers

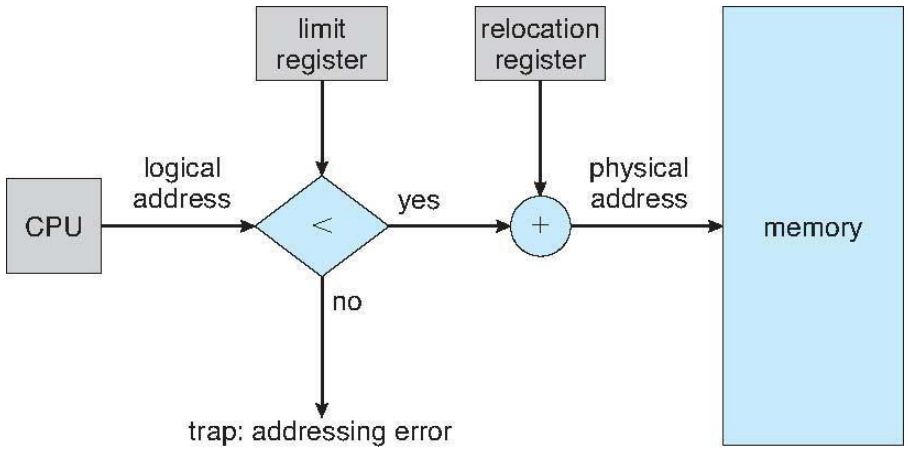


Fig 3.6 Hardware Support for Relocation and Limit Registers

Multiple-partition allocation

- ❑ Degree of multiprogramming limited by number of partitions

- **Memory Division Methods:**
    1. **Fixed Partitioning:** Memory is divided into fixed-sized, pre-determined partitions. This leads to internal fragmentation.
    2. **Variable Partitioning (Dynamic):** Memory is divided based on the specific needs of the process, which reduces internal fragmentation but can cause external fragmentation.

Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.

❑ Operating system maintains information about:

a) allocated partitions b) free partitions

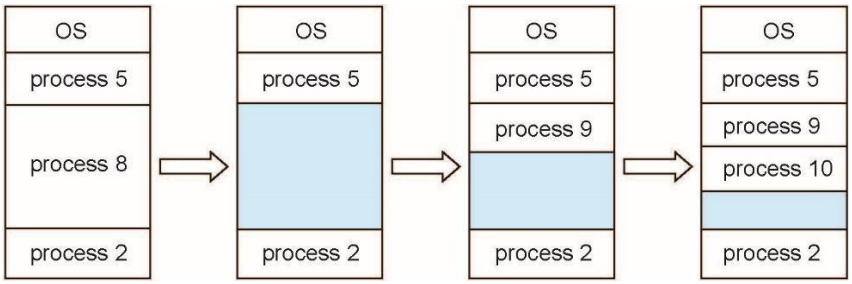


Fig 3.7 Multiple partition

Dynamic Storage-Allocation Problem

The Dynamic Storage-Allocation Problem involves efficiently managing a pool of memory, assigning blocks of varying sizes to programs at runtime (heap allocation), and reclaiming them, while minimizing fragmentation and allocation time. The core challenge is deciding which free block to use to reduce wastage, often leading to issues like memory leaks, dangling pointers, and heap fragmentation

Key Strategies for Dynamic Allocation

- **First Fit:** Allocate the first available free block that is large enough. This is usually the fastest method.
- **Best Fit:** Search the entire list for the smallest free block that is large enough, minimizing wasted space.
- **Worst Fit:** Allocate the largest available free block, leaving behind a large enough remainder that might be useful for other, smaller requests.

First fit analysis reveals that given *N* blocks allocated, 0.5 *N* blocks lost to fragmentation

1/3 may be unusable -> **50-percent rule**

Solution to external fragmentation- Compaction

Reduce external fragmentation by **compaction**

- Shuffle memory contents to place all free memory together in one large block
- Compaction is possible *only* if relocation is dynamic, and is done at **execution time**
- But there is I/O problem

  Latch job in memory while it is involved in I/O

  Do I/O only into OS buffers

Another solution to external fragmentation problem is to permit the logical address space of the process to be noncontiguous, thus allowing a process to be allocated physical memory wherever such available.
Two techniques achieve this solution: **Paging and segmentation**

SEGMENTATION

A segment is just a contiguous portion of the address space of a particular length, and in our canonical address space, we have three logically-different segments: code, stack, and heap. What segmentation allows the OS to do is to place each one of those segments in different parts of physical memory, and thus avoid filling physical memory with unused virtual address space.

Memory-management scheme that supports user view of memory
A program is a collection of segments
A segment is a logical unit such as:

- main program
- procedure
- function
- method
- object
- local variables, global variables
- common block
- stack
- symbol table
- arrays

A logical address space is a collection of segments.

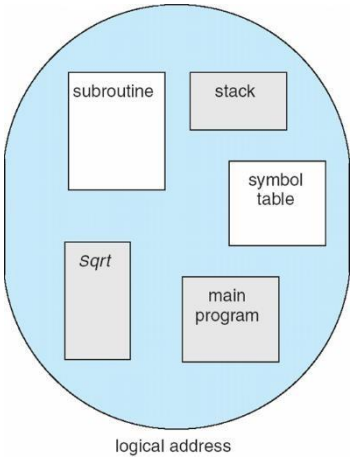


Fig 3.8 User's View of a Program

Logical View of Segmentation

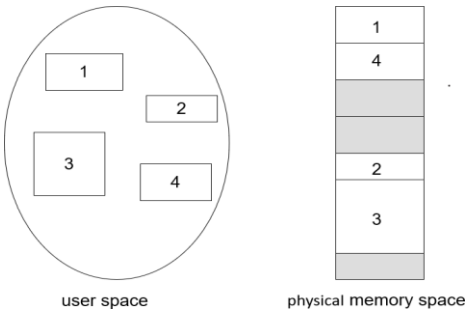


Fig 3.9 Logical View of Segmentation

GENERALIZED BASE/BOUNDS

Each segment has a name and a length. The addresses specify both the segment name and the offset within the segment. The programmer therefore specifies each address by two quantities: a segment name and an offset
For simplicity of implementation, segments are numbered and are referred to by a segment number,

rather than by a segment name. Thus, a logical address consists of a *two tuple:*
<segment-number, offset>
Although the programmer can now refer to objects in the program by a two-dimensional address, the actual physical memory is still, of course, a one-dimensional sequence of bytes. Thus, we must define an implementation to map two-dimensional user-defined addresses into one-dimensional physical addresses.
This mapping is affected by a **segment table**. Each entry in the segment table has a **segment base** and a **segment limit(bounds).**
The segment base contains the starting physical address where the segment resides in memory, and the segment limit specifies the length of the segment.

ADDRESS TRANSLATION IN SEGMENTATION

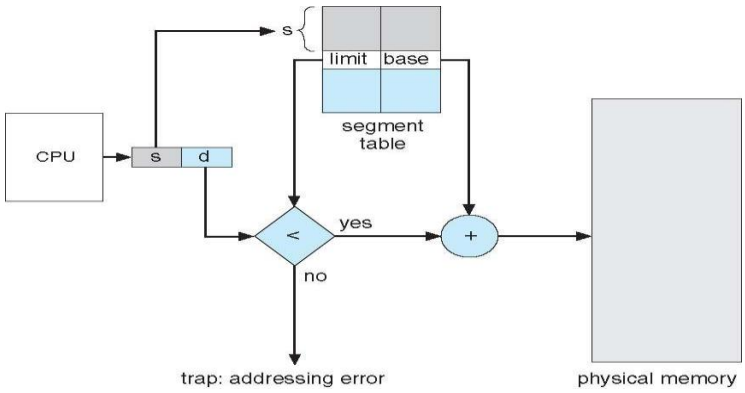


Figure 3.10 Address Translation in Segmentation

The use of a segment table is illustrated in Figure 3.10. A logical address consists of two parts: **a segment number, *s,* and an offset into that segment, *d*.** The segment number is used as an index to the segment table. **The offset *d* of the logical address must be between 0 and the segment limit.** If it is not, it's a trap to the operating system (logical addressing attempt beyond end of segment). When an offset is legal, it is added to the segment base to produce the address in physical memory of the desired byte. The segment table is thus essentially an array of base–limit (base-bounds) register pairs.

- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;
  - segment number ***s*** is legal if ***s*** < **STLR**

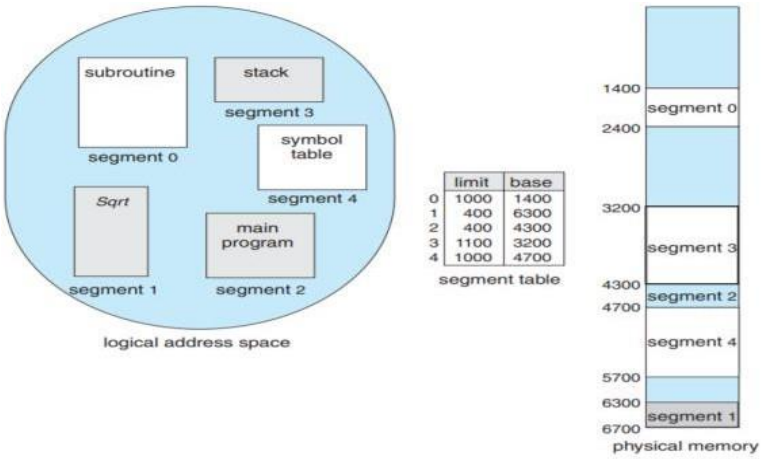


Fig 3.11 Segments in Physical Memory

Example: We have five segments numbered from 0 through 4.
The segments are stored in physical memory as shown. (Fig 3.11)

The term segmentation fault or violation arises from a memory access on a segmented machine to an illegal address.

SUPPORT FOR SHARING

Specifically, to save memory, sometimes it is useful to share certain memory segments between address spaces. In particular, code sharing is common and still in use in systems today. To support sharing, we need a little extra support from the hardware, in the form of **protection bits**. Basic support adds a few bits per segment, indicating whether or not a program can read or write a segment, or perhaps execute code that lies within the segment. By setting a code segment to read-only, the same code can be shared across multiple processes, without worry of harming isolation; while each process till thinks that it is accessing its own private memory, the OS is secretly sharing memory which cannot be modified by the process, and thus the illusion is preserved. An example of the additional information tracked by the hardware (and OS) is shown in Figure 3.12. The code segment is set to read and execute, and thus the same physical segment in memory could be mapped into multiple virtual address spaces. If a user process tries to write to a read-only page, or execute from a non-executable page, the hardware should raise an exception, and thus let the OS deal with the offending process.

Segment	Base	Size	Grows Positive?	Protection
Code	32K	2K	1	Read-Execute
Heap	34K	2K	1	Read-Write
Stack	28K	2K	0	Read-Write

Fig 3.12: Segment Register Values (with Protection)

VIRTUAL MEMORY

- Goal of various memory-management strategies used in computer systems?
- **Definition:**
  Virtual memory is a technique that allows the execution of processes that are not completely in memory.
- Virtual memory abstracts main memory into an extremely large, uniform array of storage.
- It separates the logical memory as viewed by the user from physical memory.
- This technique frees programmers from the concerns of memory-storage limitations.
- Virtual memory is a memory management technique that provides an "idealized abstraction of the storage resources that are actually available on a given machine" which "creates the abstraction to the users of a very large primary memory."

Disadvantages of conventional memory management

- The requirement that instructions must be in physical memory to be executed limits the size of a program to the size of physical memory.
- Programs often have code **which is almost never executed**
  Eg: Error code, large data structures like arrays, list are allocated more memory than they need.
- Entire program code may not **all be needed at same time**

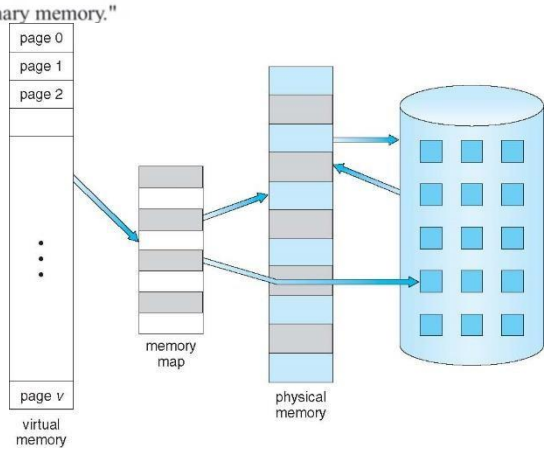


Fig 3.13 Virtual memory

Virtual address space

- The **virtual address space** of a process refers to the **logical (or virtual) view of how a process is stored in physical memory.**
- Typically, this view is that a process begins at a certain logical address-say, address 0-and exists in contiguous memory.
- Practically. physical memory may be organized in page frames and that the physical page frames assigned to a process may not be contiguous.
- It is up to the memory management unit (MMU) to map logical pages to physical page frames in memory.

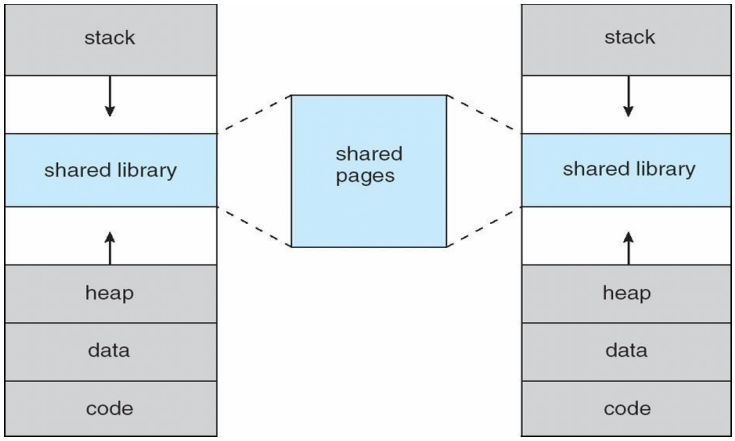


Fig 3.14 Shared Library Using Virtual memory

- Virtual memory can be implemented via:

  1. Demand paging: With demand-paged virtual memory, pages are loaded only when they are demanded during program execution. Pages that are never accessed are thus never loaded into physical memory

  The hardware to support demand paging is the same as the hardware for paging and swapping:

  - Page table. This table has the ability to mark an entry invalid through a valid–invalid bit or a special value of protection bits.
  - Secondary memory. This memory holds those pages that are not present in main memory. The secondary memory is usually a high-speed disk. It is known as the swap device, and the section of disk used for this purpose is known as swap space

  Demand Paging

  - Could bring entire process into memory at load time
  - Or bring a page into memory only when it is needed
  - Less I/O needed, no unnecessary I/O
  - Less memory needed
  - Faster response
  - More users
  - Similar to paging system with swapping (diagram on right)
  - Page is needed - reference to it
  - invalid reference - abort
  - not-in-memory - bring to memory

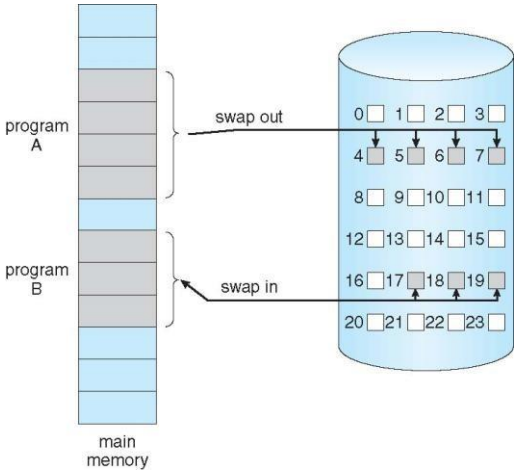


Fig 3.15 Demand Paging

  - **Lazy swapper** – never swaps a page into memory unless page will be needed
  - Swapper that deals with pages is a **pager**

2. Demand segmentation: Segmentation done upon demand.

PAGING

Paging is a memory-management scheme that permits the physical address space of a process to be noncontiguous.
Paging avoids external fragmentation and the need for compaction.

Basic Method

- Paging involves breaking physical memory into fixed-sized blocks called frames and breaking logical memory into blocks of the same size called pages.
- When a process is to be executed, its pages are loaded into any available memory frames from their source (a file system or the backing store).
- The backing store is divided into fixed-sized blocks that are of the same size as the memory frames or clusters of multiple frames.

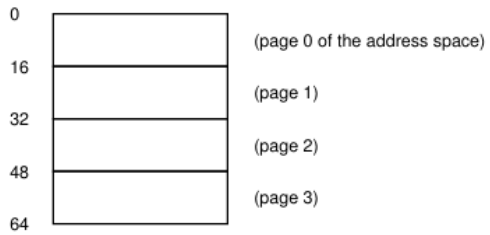


Figure 3.16: A Simple 64-byte Address Space

Paging - Address Translation Scheme

Address generated by CPU is divided into:
Page number (***p***) – used as an index into a **page table** which contains base address of each page in physical memory
Page offset (***d***) – combined with base address to define the physical memory address that is sent to the memory unit
For given logical address space 2^m and page size 2^n

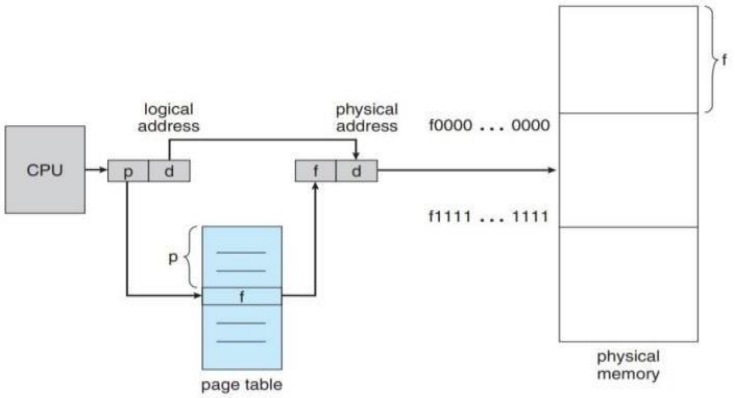
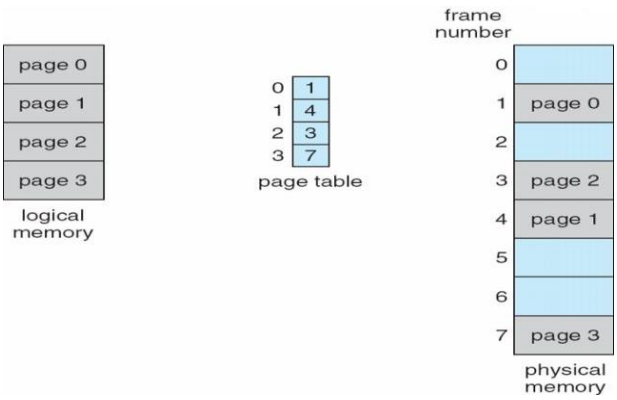


Fig 3.17 Paging Address Translation



Paging Model of Logical and Physical Memory

- The page number is used as an index into a page table.
- The page table contains the base address of each page in physical memory.
- This base address is combined with the page offset to define the physical memory address that is sent to the memory unit.

Fig 3.18 Paging Model of Logical and Physical memory

Fig 3.19 Free frames (a) before allocation and (b) after allocation.

Since the operating system is managing physical memory, it must be aware of the allocation details of physical memory—which frames are allocated, which frames are available, how many total frames there are, and so on.

- This information is generally kept in a data structure called a **frame table**.
- The frame table has one entry for each physical page frame, indicating whether the latter is free or allocated and, if it is allocated, to which page of which process or processes.

PAGE TABLE

To record where each virtual page of the address space is placed in physical memory, the operating system keeps a per-process data structure known as a page table. The major role of the page table is to store address translations for each of the virtual pages of the address space, thus letting us know where in physical memory they live.

The page table is just a data structure that is used to map virtual addresses (or really, virtual page numbers) to physical addresses (physical page numbers). The simplest form is called a linear page table, which is just

an array. The OS indexes the array by the VPN(Virtual Page Number), and looks up the page-table entry (PTE) at that index in order to find the desired PFN(Physical Frame Number).

A **valid bit** is common to indicate whether the particular translation is valid; for example, when a program starts running, it will have code and heap at one end of its address space, and the stack at the other. All the unused space in-between will be marked invalid, and if the process tries to access such memory, it will generate a trap to the OS which will likely terminate the process. Thus, the valid bit is crucial for supporting a sparse address space; by simply marking all the unused pages in the address space invalid, we remove the need to allocate physical frames for those pages and thus save a great deal of memory.

Memory protection implemented by associating protection bit with each frame to indicate if read- only or read-write access is allowed

- "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
- "invalid" indicates that the page is not in the process' logical address space

Valid (v) or Invalid (i) Bit In A Page Table

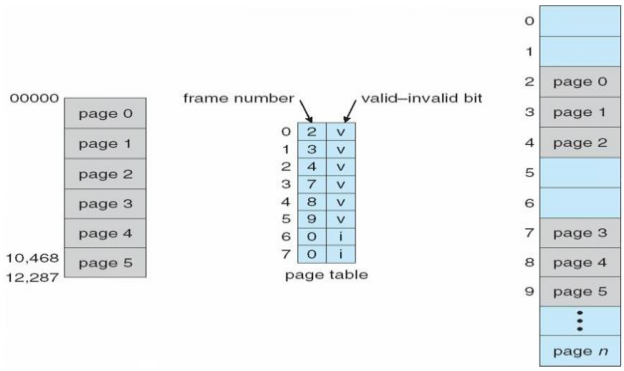


Fig 3.20: Valid (v) or Invalid (i) Bit In A Page Table

Protection bits, indicate whether the page could be read from, written to, or executed from. Again, accessing a page in a way not allowed by these bits will generate a trap to the OS.

A **present bit** indicates whether this page is in physical memory or on disk (swapped out); we will understand this in more detail when we study how to move parts of the address space to disk and back in order to support address spaces that are larger than physical memory and allow for the pages of processes that aren't actively being run to be swapped out.

A **dirty bit** is also common, indicating whether the page has been modified since it was brought into memory.

A **reference bit** is sometimes used to track whether a page has been accessed, and is useful in determining which pages are popular and thus should be kept in memory.

HARDWARE SUPPORT

The page table is implemented as a set of dedicated registers. These registers should be built with very high-

speed logic to make the paging-address translation efficient.
Every access to memory must go through the paging map, so efficiency is a major consideration. The CPU dispatcher reloads these registers, just as it reloads the other registers. The use of registers for the page table is satisfactory if the page table is reasonably small.
Most computers allow the page table to be very large (for example, 1 million entries). For these machines, the use of fast registers to implement the page table is not feasible. Rather, the page table is kept in main memory, and a page-table base register (PTBR) points to the page table. The problem with this approach is the time required to access a user memory location.

With this scheme, two memory accesses are needed to access a byte (one for the page-table entry, one for the byte).
Thus, memory access is slowed by a factor of 2. This delay would be intolerable under most circumstances. The standard solution to this problem is to use a special, small, fast lookup hardware cache called a translation look-aside buffer (TLB).

TRANSLATION LOOK-ASIDE BUFFER (TLB)

- A TLB is part of the chip's memory-management unit (MMU), and is simply a hardware cache of popular virtual-to-physical address translations; thus, a better name would be an address-translation cache. Upon each virtual memory reference, the hardware first checks the TLB to see if the desired translation is held therein; if so, the translation is performed (quickly) without having to consult the page table (which has all translations). Because of their tremendous performance impact, TLBs in a real sense make virtual memory possible
- The TLB is associative, high-speed memory. Each entry in the TLB consists of two parts: a key (or tag) and a value. When the associative memory is presented with an item, the item is compared with all keys simultaneously. If the item is found, the corresponding value field is returned.
- When a logical address is generated by the CPU, its page number is presented to the TLB. If the page number is found it is called **TLB hit** and its frame number is immediately available and is used to access memory.
- If the page number is not in the TLB (**known as a TLB miss**), a memory reference to the page table must be made.
- Depending on the CPU, this may be done automatically in hardware or via an interrupt to the operating system.
- When the frame number is obtained, we can use it to access memory.
- We add the page number and frame number to the TLB
- If the TLB is already full of entries, an existing entry must be selected for replacement. Replacement policies range from least recently used (LRU) through round-robin to random.
- Some CPUs allow the operating system to participate in LRU entry replacement, while others handle the matter themselves.
- The percentage of times that the page number of interests is found in the TLB is called the **hit ratio**. An 80- percent hit ratio, for example, means that we find the desired page number in the TLB 80 percent of the time. If it takes 100 nanoseconds to access memory, then a mapped-memory access takes 100 nanoseconds when the page number is in the TLB.

EXAMPLE: ACCESSING AN ARRAY

To make clear the operation of a TLB, let's examine a simple virtual address trace and see how a TLB can improve its performance. In this example, let's assume we have an array of 10 4-byte integers in memory, starting at virtual address 100. Assume further that we have a small 8-bit virtual address space, with 16-byte pages; thus, a virtual address breaks down into a 4-bit VPN (there are 16 virtual pages) and a 4-bit offset (there are 16 bytes on each of those pages). Figure 3.21 shows the array laid out on the 16 16-byte pages of the system. The array's first entry(a[0]) begins on(VPN=06, off set=04); only three 4-byte integers fit onto that page. The array continues onto the next page (VPN=07), where the next four entries (a[3] ... a[6]) are found. Finally, the last three entries ofthe10-entry array(a[7]...a[9]) are located on the next page of the address space (VPN=08). Now let's consider a simple loop that accesses each array element, in C:

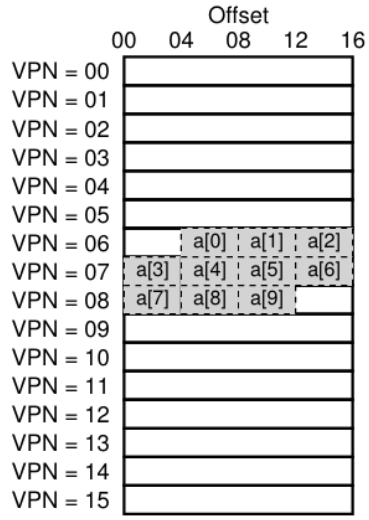


Figure 3.21: Example: An Array In A Tiny Address Space

```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```

For the sake of simplicity, we will pretend that the only memory accesses the loop generates are to the array (ignoring the variables i and sum, as well as the instructions themselves). When the first array element (a[0]) is accessed, the CPU will see a load to virtual address 100. The hardware extracts the VPN from this (VPN=06), and uses that to check the TLB for a valid translation. Assuming this is the first time the program accesses the array, the result will be a TLB miss. The next access is to a[1], and there is some good news here: a TLB hit! Because the second element of the array is packed next to the first, it lives on the same page; because we've already accessed this page when accessing the first element of the array, the translation is already loaded into the TLB. And hence the reason for our success. Access to a[2] encounters similar success (another hit), because it too lives on the same page as a[0] and a[1]. Unfortunately, when the program accesses a[3], we encounter an other TLB miss. However, once again, the next entries (a[4] ... a[6]) will hit in the TLB, as they all reside on the same page in memory. Finally, access to a[7] causes one last TLB miss. The hardware once again consults the page table to figure out the location of this virtual page in physical memory, and updates the TLB accordingly. The final two accesses (a[8] and a[9])receive the benefits of this TLB update; when the hardware looks in the TLB for their translations, two more hits result. Let us summarize TLB activity during our ten accesses to the array: miss, hit, hit, miss, hit, hit, hit, miss, hit, hit. Thus, our TLB hit rate, which is the number of hits divided by the total number of accesses, is 70%. Although this is not too high (indeed, we desire hit rates that approach 100%), it is non-zero, which may be a surprise. Even though this is the first time the program accesses the array, TLB performance gains benefit from spatial locality. The elements of the array are packed tightly into pages (i.e., they are close to one another in space), and thus only the first access to an element on a page yields a TLB miss. Also note the role

that page size plays in this example. If the page size had simply been twice as big (32 bytes, not 16), the array access would suffer even fewer misses. As typical page sizes are more like 4KB, these types of dense, array-based accesses achieve excellent TLB performance, encountering only a single miss per page of accesses. One last point about TLB performance: if the program, soon after this loop completes, accesses the array again, we'd likely see an even better result, assuming that we have a big enough TLB to cache the needed translations: hit, hit, hit, hit, hit, hit, hit, hit, hit, hit. In this case, the TLB hit rate would be high because of temporal locality, i.e., the quick re-referencing of memory items in time. Like any cache, TLBs rely upon both spatial and temporal locality for success, which are program proper ties. If the program of interest exhibits such locality (and many programs do), the TLB hit rate will likely be high.

The idea behind hardware caches is to take advantage of locality in instruction and data references.

There are usually two types of locality: temporal locality and spatial locality.

With temporal locality, the idea is that an instruction or data item that has been recently accessed will likely be re-accessed soon in the future.

With spatial locality, the idea is that if a program accesses memory at address x, it will likely soon access memory near x.

HANDLING TLB MISS

TLB Miss is handled by the **hardware, or the software (OS)**. In the olden days, the hardware had complex instruction sets (sometimes called CISC, for complex-instruction set computers). Thus, the hardware would handle the TLB miss entirely. To do this, the hardware has to know exactly where the page tables are located in memory (via a page table base register as well as their exact format; on a miss, the hardware would "walk" the page table, find the correct page-table entry and extract the desired translation, update the TLB with the translation, and retry the instruction. An example of an "older" architecture that has hardware-managed TLBs is the Intel x86 architecture, which uses a fixed multi-level page table.

More modern architectures have what is known as a software-managed TLB. On a TLB miss, the hardware simply raises an exception, which pauses the current instruction stream, raises the privilege level to kernel mode, and jumps to a trap handler.

TLB STRUCTURE

Paging Hardware With TLB

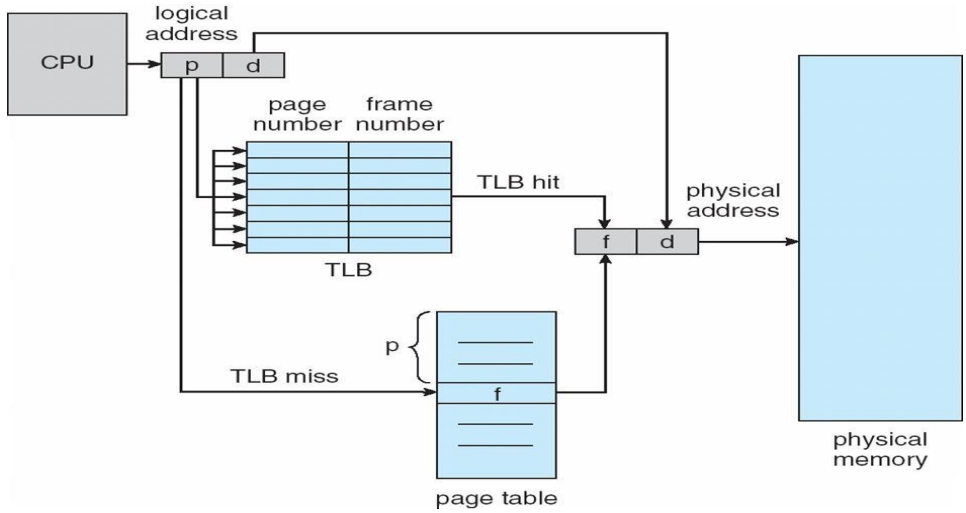


Fig 3:22 Paging Hardware With TLB

The TLB commonly has a **valid bit**, which says whether the entry has a valid translation or not.
Also common are **protection bits**, which determine how a page can be accessed (as in the page table). For example, code pages might be marked read and execute, whereas heap pages might be marked read and write.
To reduce this overhead, some systems add hardware support to enable sharing of the TLB across context switches. In particular, some hard ware systems provide an **address space identifier (ASID)** field in the TLB. You can think of the ASID as a process identifier (PID), but usually it has fewer bits (e.g., 8 bits for the ASID versus 32 bits for a PID).

VPN	PFN	valid	prot	ASID
10	100	1	rwx	1
—	—	0	—	—
10	170	1	rwx	2
—	—	0	—	—

Fig 3.23 TLB contents

VPN-Virtual Page Number (Virtual Address)
PFN- Physical Frame Number (Physical Address)
Global bit (G), which is used for pages that are globally-shared among processes. Thus, if the global bit is set, the ASID is ignored.
Coherence (C)bits, which determine how a page is cached by the hardware (a bit beyond the scope of these notes);
A dirty bit which is marked when the page has been modified.
There is also a **page mask field** (not shown), which supports multiple page sizes;

REDUCING THE PAGE TABLE SIZE

1. Hybrid Approach: Paging and Segments

Instead of having a single page table for the entire address space of the process, in this approach, it is having one per logical segment. In this example, we might thus have three-page tables, one for the code, heap, and stack parts of the address space. Now, remember with segmentation, we had a base register where each segment lived in physical memory, and a bound or limit register that told us the size of said segment. In hybrid, we use the base not to point to the segment itself but rather to hold the physical address of the page table of that segment. The bounds register is used to indicate the end of the page table (i.e., how many valid pages it has).
Assume a 32-bit virtual address space with 4KB pages, and an address space split into four segments. We'll only use three segments for this example: one for code, one for heap, and one for stack. To determine which segment an address refers to, we'll use the top two bits of the address space. Let's assume 00 is the unused segment, with 01 for code, 10 for the heap, and 11 for the stack. Thus, a virtual address looks like this:

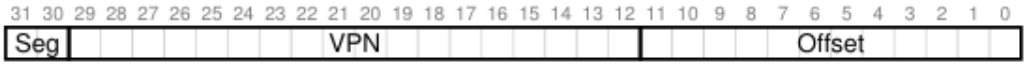


Fig 3.24 Hybrid approach

In the hardware, assume that there are thus three base/bounds pairs, one each for code, heap, and stack. When a process is running, the base register for each of these segments contains the physical address of a linear page table for that segment; thus, each process in the system now has three page tables associated with it. On a context switch, these registers must be changed to reflect the location of the page tables of the newly

running process. On a TLB miss (assuming a hardware-managed TLB, i.e., where the hardware is responsible for handling TLB misses), the hardware uses the segment bits (SN) to determine which base and bounds pair to use.

2. **Multi-level Page Tables**

The basic idea behind a multi-level page table is simple. First, chop up the page table into page-sized units; then, if an entire page of page-table entries (PTEs) is invalid, don't allocate that page of the page table at all. To track whether a page of the page table is valid (and if valid, where it is in memory), use a new structure, called the page directory. The page directory thus either can be used to tell you where a page of the page table is, or that the entire page of the page table contains no valid pages. Figure 3.25 shows an example. On the left of the figure is the classic linear page table; even though most of the middle regions of the address space are not valid, we still have to have page-table space allocated for those regions (i.e., the middle two pages of the page table). On the right is a multi-level page table. The page directory marks just two pages of the page table as valid (the first and last); thus, just those two pages of the page table reside in memory. It just makes parts of the linear page table disappear (freeing those frames for other uses), and tracks which pages of the page table are allocated with the page directory. The page directory, in a simple two-level table, contains one entry per page of the page table. It consists of a number of page directory entries (PDE). A PDE (minimally) has a valid bit and a page frame number (PFN), similar to a PTE. However, the meaning of this valid bit is slightly different: if the PDE entry is valid, it means that at least one of the pages of the page table that the entry points to (via the PFN)is valid, i.e., in at least one PTE on that page pointed to by this PDE, the valid bit in that PTE is set to one. If the PDE entry is not valid (i.e., equal to zero), the rest of the PDE is not defined.

Multi-level page tables have some obvious advantages.

- First, the multi-level table only allocates page-table space in proportion to the amount of address space you are using; thus, it is generally compact and supports sparse address spaces.
- Second, if carefully constructed, each portion of the page table fits neatly within a page, making it easier to manage memory; the OS can simply grab the next free page when it needs to allocate or grow a page table.

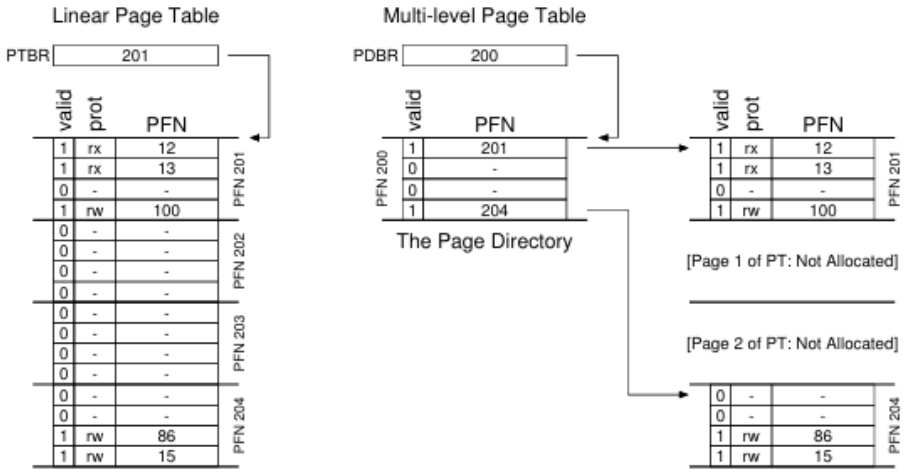


Figure 3.25: Linear (Left) And Multi-Level (Right) Page Tables

3. Inverted Page Tables

An even more extreme space savings in the world of page tables is found with inverted page tables. Here, instead of having many page tables (one per process of the system), we keep a single page table that has an entry for each physical page of the system. The entry tells us which process is using this page, and which virtual page of that process maps to this physical page.

INVERTED PAGETABLES – ADVANTAGES AND DISADVANTAGES

- This scheme **decreases the amount of memory needed to store each page table.**
- It increases the amount of time needed to search the table when a page reference occurs.
- The **inverted page table is sorted by physical address, but lookups occur on virtual addresses**, the whole table might need to be searched for a match and this search would take long.

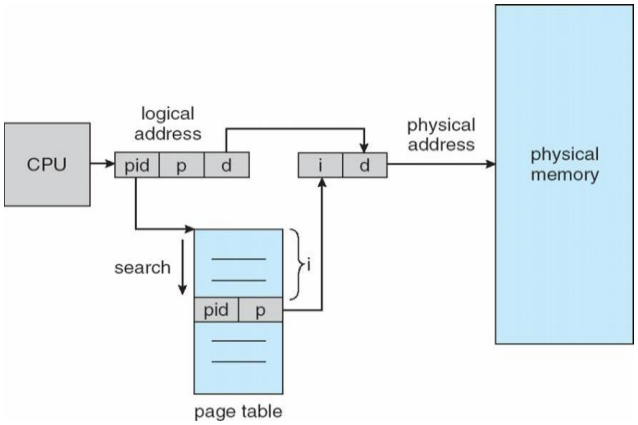


Fig 3.26 Inverted Page Table

4. Swapping the Page Tables to Disk

Even with our many tricks to reduce the size of page tables, it is still possible, however, that they may be too big to fit into memory all at once. Thus, some systems place such page tables in kernel virtual memory, thereby allowing the system to swap some of these page tables to disk when memory pressure gets a little tight.

Effective Access Time

An 80-percent hit ratio means that we find the desired page number in the associative registers 80 percent of the time. If it takes 20 nanoseconds to search the TLB, and 100 nanoseconds to access memory, then a mapped memory access takes 120 nanoseconds when the page number is in the TLB.

If we fail to find the page number in the TLB (20 nanoseconds, TLB Miss), then we must first access memory for the page table and frame number (100 nanoseconds), and then access the desired byte in memory (100 nanoseconds), for a total of 220 nanoseconds.

To find the *effective access time,* we must weigh each case by it probability:

Effective Access Time=0.80*120+0.20*220
= 140 nanoseconds.

Consider a paging system with the page table stored in memory.

(a) If a memory reference takes 200 nanoseconds, how long does a paged memory reference take?

(b) If we add associative registers, and 75% of all page-table references are found in the associative registers, what is the effective memory reference time? (Assume that finding a page-table entry in the associative registers takes zero time, if the entry is there.)

- **400 nanoseconds. 200 ns to access the page table plus 200 ns to access the word in memory.**
- **250 nanoseconds. 75% of the time it's 200 ns, and the other 25% of the time it's 400ns, so the equation is: EAT = (.75*200)+(.25*400)**

- Associative Lookup Time = ε time units
- Hit ratio = α
  - Hit ratio – percentage of times that a page number is found in the TLB; ratio related to number of TLB
- Consider hit ratio α, TLB search time ε, and memory access time m
  - Effective Access Time (EAT)

$$EAT = (m + \varepsilon)\alpha + (2m + \varepsilon)(1 - \alpha)$$
$$= 2m + \varepsilon - m\alpha$$

- Consider more realistic hit ratio $\alpha = 99\%$, ε = 20ns for TLB search, m = 100ns for memory access
  - EAT = 0.99 × (100 + 20) + 0.01 × (200 + 20) = 121ns (21% slowdown)

GOING BEYOND PHYSICAL MEMORY

SWAP SPACE

A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution.
Total physical memory space of processes can exceed physical memory
Backing store – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
Roll out, roll in – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped.
System maintains a ready queue of ready-to-run processes which have memory images on disk.

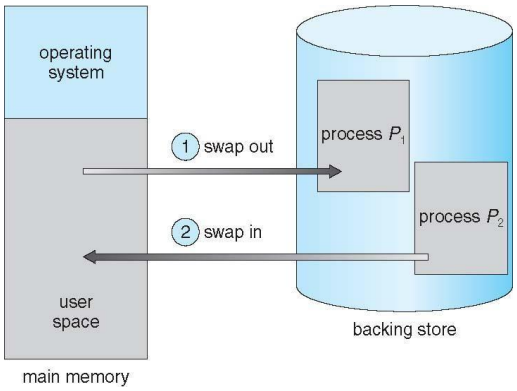


Fig 3.27 Schematic View of Swapping

Context Switch Time including Swapping

- Context-switch time in swapping is very high
  - Assume swapping 100MB to backing store with a transfer rate of 50MB/sec
    - Swap out time of a process is 2000 ms **(= 2 seconds)**
    - Swap in time of a same sized process is also 2000 ms
    - Total context-switch swapping component time is **4000ms (= 4 seconds)**
  - We have ignored other components of context-switch time and ignored other disk performance aspect. **But, the major part of swap time is transfer time**
- Total transfer time is directly proportional to the amount of memory swapped

PAGE FAULT

Access to a page marked invalid causes a page fault. The paging hardware, in translating the address

through the page table, will notice that the invalid bit is set, causing a trap to the operating system. This trap is the result of the operating system's failure to bring the desired page into memory.

With TLB misses, we have two types of systems: hardware managed TLBs (where the hardware looks in the page table to find the desired translation) and software-managed TLBs (where the OS does). In either type of system, if a page is not present, the OS is put in charge to handle the page fault. The appropriately-named OS page-fault handler runs to determine what to do. Virtually all systems handle page faults in software; even with a hardware-managed TLB, the hardware trusts the OS to manage this important duty. If a page is not present and has been swapped to disk, the OS will need to swap the page into memory in order to service the page fault. Thus, a question arises: how will the OS know where to find the desired page? In many systems, the page table is a natural place to store such information. Thus, the OS could use the bits in the PTE normally used for data such as the PFN of the page for a disk address. When the OS receives a page fault for a page, it looks in the PTE to find the address, and issues the request to disk to fetch the page into memory.

When the disk I/O completes, the OS will then update the page table to mark the page as present, update the PFN field of the page-table entry (PTE) to record the in-memory location of the newly-fetched page, and retry the instruction. This next attempt may generate a TLB miss, which would then be serviced and update the TLB with the translation (one could alternately update the TLB upon when servicing the page fault, to avoid this step). Finally, a last restart would find the translation in the TLB and thus proceed to fetch the desired data or instruction from memory at the translated physical address. Note that while the I/O is in flight, the process will be in the blocked state. Thus, the OS will be free to run other ready processes while the page fault is being serviced. Because I/O is expensive, this overlap of the I/O (page fault) of one process and the execution of another is yet another way a multi-programmed system can make the most effective use of its hardware.

PAGE FAULT CONTROL FLOW

- There are now three important cases to understand when a TLB miss occurs. **First, that the page was both present and valid.** In this case, the TLB miss handler can simply grab the PFN from the PTE, retry the instruction (this time resulting in a TLB hit), and thus continue as described (many times) before.
- In the second case, the page fault handler must be run; although this was a legitimate page for the process to access (it is **valid**, after all), it is **not present in physical memory.**
- Third (and finally), the access could be to an **invalid page**, due for example to a bug in the program.
- In this case, no other bits in the PTE really matter; the hardware traps this invalid access, and the OS trap handler runs, likely terminating the offending process.
- From the software control flow in, we can see what the OS roughly must do in order to service the page fault. First, the OS must find a physical frame for the soon-to-be-faulted-in page to reside within; if there is no such page, we'll have to wait for the replacement algorithm to run and kick some pages out of memory, thus freeing them for use here.
- With a physical frame in hand, the handler then issues the I/O request to read in the page from swap space. Finally, when that slow operation completes, the OS updates the page table and

retries the instruction. The retry will result in a TLB miss, and then, upon another retry, a TLB hit, at which point the hardware will be able to access the desired item.

The procedure for handling this page fault is straightforward :

1. **We check an internal table (usually kept with the process control block) for this process to determine whether the reference was a valid or an invalid memory access.**
2. **If the reference was invalid, we terminate the process. If it was valid but we have not yet brought in that page, we now page it in.**
3. **We find a free frame (by taking one from the free-frame list, for example).**
4. **We schedule a disk operation to read the desired page into the newly allocated frame.**
5. **When the disk read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.**
6. **We restart the instruction that was interrupted by the trap. The process can now access the page as though it had always been in memory.**

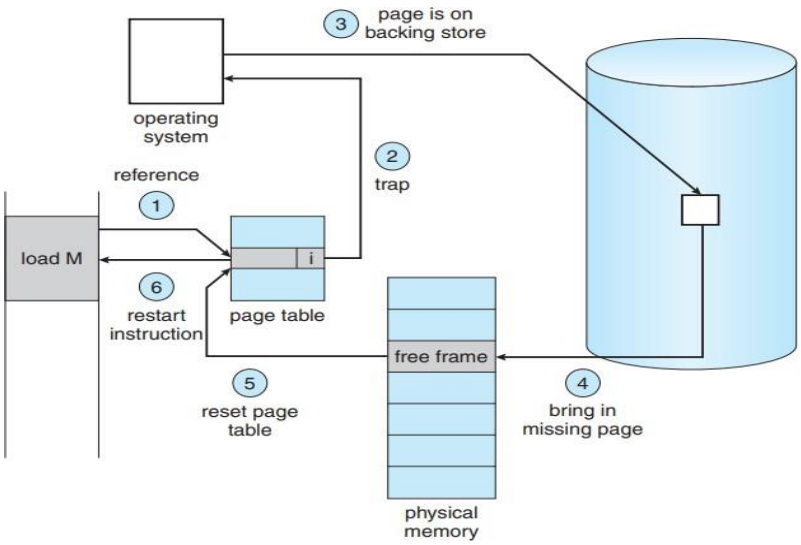


Fig 3.28 Steps in handling a page fault

PAGE REPLACEMENT POLICIES

Need For Page Replacement

Prevent **over-allocation** of memory by modifying page-fault service routine to include page replacement. Use **modify** (**dirty**) **bit** to reduce overhead of page transfers – only modified pages are written to disk. Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory.

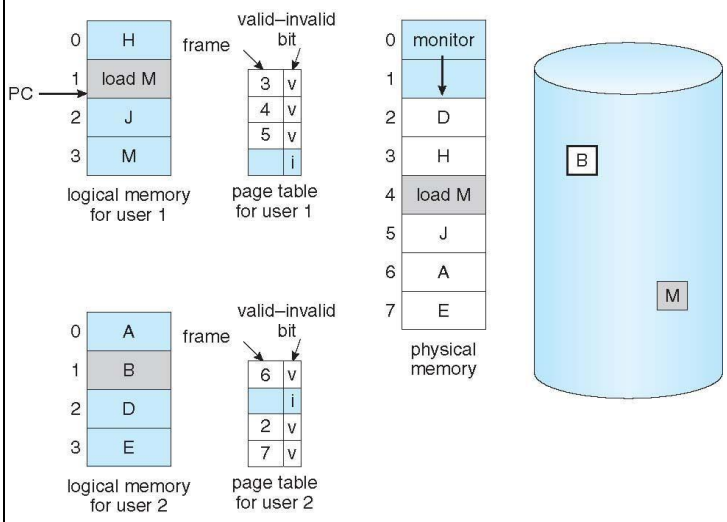


Fig 3.29 Need for page replacement

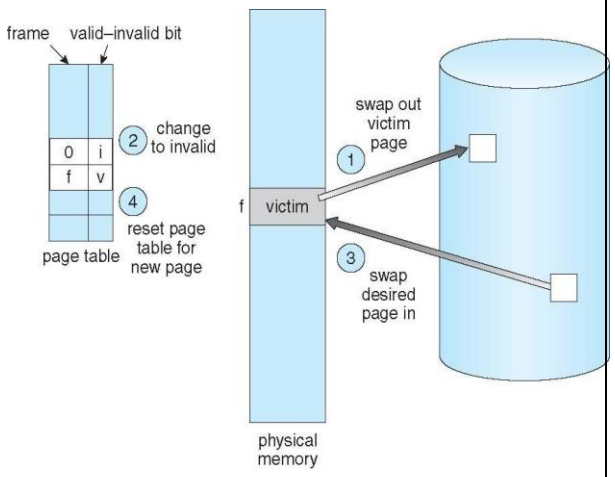


Fig 3.30 Basic Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame: If there is a free frame, use it. If there is no free frame, use a page replacement algorithm to select a **victim frame.** Write victim frame to disk if dirty.
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

If the present bit is set to one, it means the page is present in physical memory and everything proceeds as above; if it is set to zero, the page is not in memory but rather on disk somewhere. The act of accessing a page that is not in physical memory is commonly referred to as a page fault.

Page Replacement Algorithms

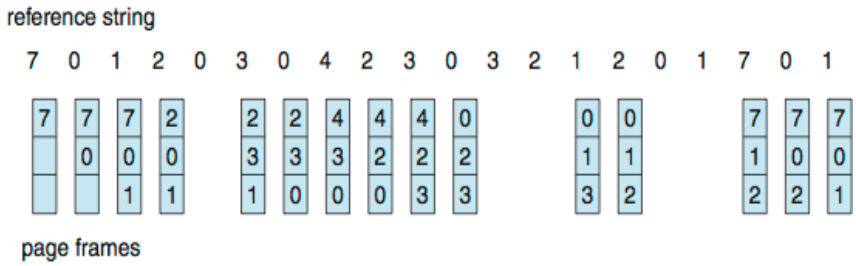
1. **First-in, first-out page replacement**
2. **Least Recently Used page replacement**
3. **Optimal page replacement algorithm**
4. **Random page replacement algorithm**
5. **Counting-Based Page Replacement**
   - **Least frequently used (LFU)**
   - **Most frequently used (MFU)**

1. First-In-First-Out (FIFO) Algorithm

- The operating system maintains a list of all pages currently in memory, with the page at the head of the list the oldest one and the page at the tail the most recently arrived.
- On a page fault, remove the oldest, i.e. head of the line and the new page added to the tail of the list.
- Easy to implement

- Oldest might be most heavily used!

Example: Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1**



Page faults=15
No.of Hits=5
Hit ratio=5/20
Miss Ratio=15/20

- 3 frames (3 pages can be in memory at a time per process)
- How to track ages of pages? Just use a FIFO queue
- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
- Adding more frames can cause more page faults!
- **Belady's anomaly** is the phenomenon in which increasing the number of page frames results in an increase in the number of page faults for certain memory access patterns. This phenomenon is commonly experienced when using the first-in first-out (FIFO) page replacement algorithm.

> Belady (of the optimal policy) and colleagues found an interesting reference stream that behaved a little unexpectedly [BNS69]. The memory-reference stream: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. The replacement policy they were studying was FIFO. The interesting part: how the cache hit rate changed when moving from a cache size of 3 to 4 pages.
>
> In general, you would expect the cache hit rate to *increase* (get better) when the cache gets larger. But in this case, with FIFO, it gets worse! Calculate the hits and misses yourself and see. This odd behavior is generally referred to as **Belady's Anomaly** (to the chagrin of his co-authors).

- FIFO and Random clearly do not obey the stack property, and thus are susceptible to anomalous behavior.

2. Least Recently Used (LRU) Algorithm

Use past knowledge rather than future

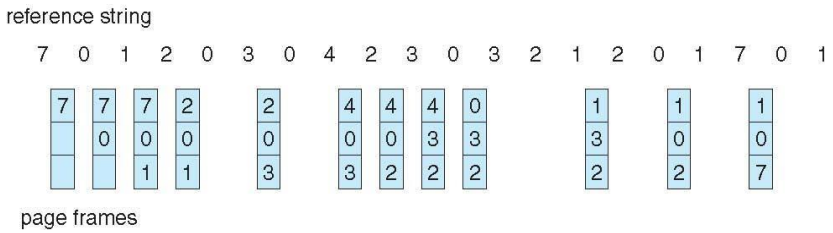
Replace page that has not been used in the most amount of time

Associate time of last use with each page

12 faults – better than FIFO but worse than OPT

- Generally good algorithm and frequently used
- But how to implement?

Example:



Page faults=12
No.of Hits=8
Hit ratio=8/20
Miss Ratio=12/20

Stack implementation

Whenever a page is referenced, it is removed from the stack and put on the top. In this way, the most recently used page is always at the top of the stack and the least recently used page is always at the bottom

LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly

LRU-Approximation Page Replacement

- **Additional-Reference-Bits Algorithm**
- Additional ordering information by recording the reference bits at regular intervals.
- We can keep an 8-bit byte for each page in a table.
- At regular intervals (say, every 100 milliseconds), a timer interrupt transfers control to the operating system.
- The operating system shifts the reference bit for each page into the high-order bit of its 8-bit byte, shifting the other bits right by 1 bit and discarding the low-order bit.
- These 8-bit shift registers contain the history of page use for the last eight time periods.
- If the shift register contains 00000000, for example, then the page has not been used for eight time periods.
- A page that is used at least once in each period has a shift register value of 11111111.
- A page with a history register value of 11000100 has been used more recently than one with a value of 01110111.
- **Second-Chance Algorithm**
- The basic algorithm of second-chance replacement is a FIFO replacement algorithm.
- When a page has been selected, however, we inspect its reference bit.
- If the value is 0, we proceed to replace this page; but if the reference bit is set to 1, we give the page a second chance and move on to select the next FIFO page.
- When a page gets a second chance, its reference bit is cleared, and its arrival time is reset to the current time
- A page that is given a second chance will not be replaced until all other pages have been replaced (or given second chances).

- In addition, if a page is used often enough to keep its reference bit set, it will never be replaced.
- One way to implement the second-chance algorithm (sometimes referred to as the **clock algorithm) is as a circular queue.**
- **A pointer (that is, a hand on** the clock) indicates which page is to be replaced next. When a frame is needed, the pointer advances until it finds a page with a 0 reference bit.
- As it advances , it clears the reference bit.
- Once a victim page is found, the page is replaced, and the new page is inserted in the circular queue in that position.
- Notice that, in the worst case, when all bits are set, the pointer cycles through the whole queue, giving each page a second chance. It clears all the reference bits before selecting the next page for replacement.

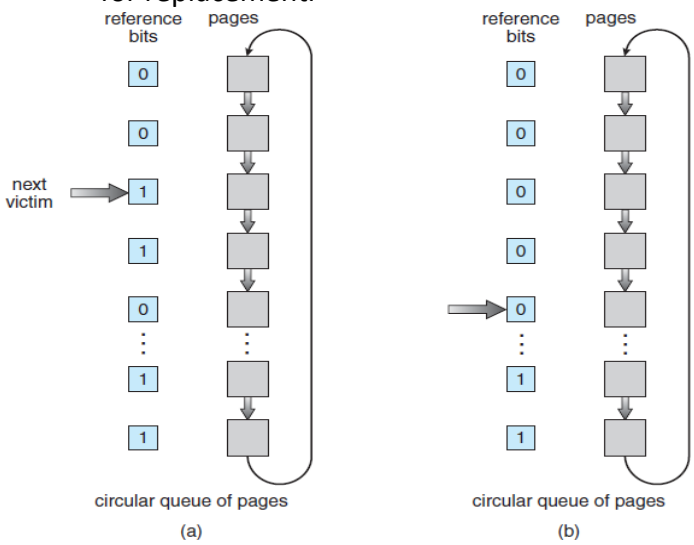


Figure Second-chance (clock) page-replacement algorithm.

Enhanced Second-Chance Algorithm

- We can enhance the second-chance algorithm by considering the reference bit and the modify bit as an ordered pair.
- With these two bits, we have the following four possible classes:
  - **(0, 0) neither recently used nor modified—best page to replace**
  - **(0, 1) not recently used but modified—not quite as good, because the** page will need to be written out before replacement
  - **(1, 0) recently used but clean—probably will be used again soon**
  - **(1, 1) recently used and modified—probably will be used again soon, and** the page will be need to be written out to disk before it can be replaced.
  - Each page is in one of these four classes.
  - When page replacement is called for, we use the same scheme as in the clock algorithm; but instead of examining whether the page to which we are pointing has the reference bit set to 1, we examine the

class to which that page belongs. We replace the first page encountered in the lowest nonempty class. Notice that we may have to scan the circular queue several times before we find a page to be replaced.

3.

Optimal Page Replacement

- An optimal page replacement algorithm has the lowest page-fault rate of all algorithms and will never suffer from Belady's anomaly
- Such an algorithm does exist and has been called OPT or MIN. It is simply this:

  Replace the page that will not be used for the longest period of time
- Use of this page-replacement algorithm guarantees the lowest possible page-fault rate for a fixed number of pages

- Reference String is: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

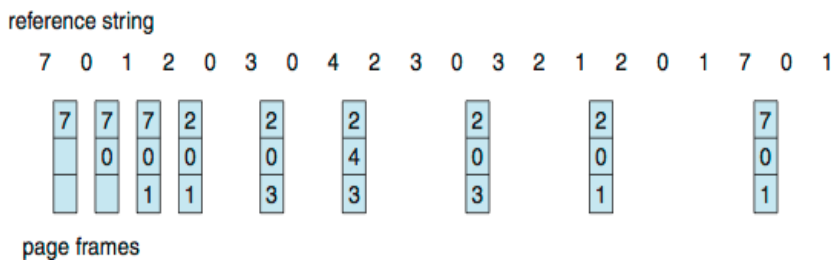
1	1	1	1	1	3	3
	2	2	2	2	2	4
		3	4	5	5	5

Total Page Fault are : 7

- Page Fault : 1+1+1+1+1+1+1

Check *page that will not be used for the longest period of time. And replace it if it not present in frame*

Example:



Page faults=9
No.of Hits=11
Hit ratio=11/20
Miss Ratio=9/20

4) Random Page Replacement Algorithm

Another similar replacement policy is Random, which simply picks a random page to replace under memory pressure. Random has properties similar to FIFO; it is simple to implement, but it doesn't really try to be too intelligent in picking which blocks to evict. Random is as good as optimal, achieving less hits; sometimes it does much worse. How Random does depends on the luck of the draw.

Example:

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	0	1, 2, 3
0	Miss	1	2, 3, 0
3	Hit		2, 3, 0
1	Miss	3	2, 0, 1
2	Hit		2, 0, 1
1	Hit		2, 0, 1

5.Counting-Based Page Replacement

- The **least frequently used (LFU) page-replacement algorithm requires that** the page with the smallest count be replaced. The reason for this selection is that an actively used page should have a large reference count. A problem arises, however, when a page is used heavily during the initial phase of a process but then is never used again. Since it was used heavily, it has a large count and remains in memory even though it is no longer needed. One solution is to shift the counts right by 1 bit at regular intervals, forming an exponentially decaying average usage count.

- The **most frequently used (MFU) page-replacement algorithm is based** on the argument that the page with the smallest count was probably just brought in and has yet to be used.

THRASHING

What should the OS do when memory is simply oversubscribed, and the memory demands of the set of running processes simply exceeds the available physical memory?
In this case, the system will constantly be paging, a condition sometimes referred to as thrashing.
Thrashing in an operating system is a performance collapse occurring when the system spends more time swapping pages between RAM and disk than executing actual processes. It is caused by excessive page faults, typically due to too many active processes fighting for insufficient memory.
Some earlier operating systems had a fairly sophisticated set of mechanisms to both detect and cope with thrashing when it took place. For example, given a set of processes, a system could decide not to run a subset of processes, with the hope that the reduced set of processes working sets (the pages that they are using actively) fit in memory and thus can make progress. This approach, generally known as **admission control**, states that it is sometimes better to do less work well than to try to do everything at once poorly, a situation we often encounter in real life as well as in modern computer systems (sadly). Some current systems take more a draconian approach to memory overload. For example, some versions of Linux run an out-of-memory killer when memory is oversubscribed; this daemon chooses a memory intensive process and kills it, thus reducing memory in a none-too-subtle manner. While successful at reducing memory pressure, this approach can have problems, if, for example, it kills the X server and thus renders any applications requiring the display unusable.

Key Aspects of Thrashing

- **Cause:** When the active set of pages (working set) for all running processes exceeds the available physical memory (RAM), the OS is forced to constantly move pages in and out of swap space.
- **The Cycle:** A high degree of multiprogramming causes a high page fault rate. The OS, attempting to increase CPU usage, brings in more processes, which only increases the demand for memory, exacerbating the problem
- **Symptoms:**
  - **High Page Fault Rates:** Constant interruptions to fetch data from disk.
  - **Low CPU Utilization:** The CPU sits idle waiting for disk I/O to complete.
  - **Extremely Slow Performance:** System responsiveness drops to near zero.
  - **High Disk Activity:** The hard drive light is consistently blinking.

Prevention and Mitigation

- **Working Set Model:** The OS monitors the active pages (working set) of each process. If a process does not have enough frames to hold its working set, the system suspends it to free up memory.
- **Page-Fault Frequency (PFF):** A more direct technique where the system monitors the rate of page faults. If the rate is too high, the process is given more frames; if too low, frames are taken away.
- **Reducing Multiprogramming:** Temporarily reducing the number of active processes allows the remaining processes to run efficiently.
- **Increasing Physical Memory:** The most direct, albeit expensive, hardware solution.

Thrashing differs from high CPU usage in that the latter often indicates high productive work, whereas thrashing indicates that almost all resources are being used for data movement.

****